

ESCUELA POLITECNICA NACIONAL



SISTEMAS EMBEBIDOS

GEMELO DIGITAL DE BRAZO ROBÓTICO CON
SENSORES INERCIALES Y APLICACIÓN WEB

INTEGRANTES: SEBASTIAN AISALLA,
TATIANA GUALPA, JHOEL NARVÁEZ,
FRANCIS VELASTEGUI, STALIN YUNGAN

GERMAN EDUARDO RODRIGUEZ GALAN

2026 – A

1. Resumen Ejecutivo

El presente proyecto desarrolla un gemelo digital de bajo costo de un brazo humano, capaz de capturar y replicar movimientos en tiempo real mediante sensores inerciales y una interfaz web tridimensional. El sistema emplea tres sensores MPU6050 ubicados en hombro, codo y muñeca, conectados a un microcontrolador ESP32 a través de un multiplexor I2C TCA9548A. Los datos de orientación son transmitidos por puerto serial a un puente local desarrollado en Node.js, que los publica mediante protocolo WebSocket a una aplicación web construida con React y Three.js, donde un modelo 3D del brazo replica los movimientos en tiempo real con una latencia menor a 50 ms y una frecuencia de actualización de 100 Hz.

2. Introducción

En la actualidad, la tecnología permite crear representaciones virtuales de objetos y sistemas reales para observar su comportamiento sin necesidad de interactuar directamente con ellos. Estas representaciones, conocidas como gemelos digitales, se han convertido en una herramienta útil en diferentes áreas como la educación, la medicina, la industria y la investigación.

Uno de los campos donde esta tecnología tiene gran potencial es la captura y visualización de movimientos humanos. Gracias a ella es posible observar y analizar movimientos en tiempo real, lo que resulta útil en aplicaciones como la rehabilitación física, el control de robots, la realidad virtual y los procesos de entrenamiento.

Sin embargo, muchas de las soluciones existentes utilizan equipos especializados y costosos, lo que dificulta su implementación en proyectos académicos o de bajo presupuesto. Por esta razón, surge la necesidad de desarrollar alternativas más accesibles que permitan obtener resultados similares utilizando componentes económicos y de fácil adquisición.

En este proyecto se desarrolló un gemelo digital de un brazo humano utilizando un microcontrolador ESP32 y sensores inerciales MPU6050. El sistema captura los movimientos del brazo y los representa en una aplicación web tridimensional en tiempo real. De esta manera, se demuestra que es posible implementar una solución funcional y de bajo costo que puede servir como base para futuras aplicaciones en educación, investigación y automatización.

3. Planteamiento del Problema

Problema

La captura de movimientos humanos en tiempo real para su replicación en sistemas virtuales representa un desafío significativo en aplicaciones de teleoperación, rehabilitación física y realidad virtual. Los sistemas comerciales convencionales utilizan tecnologías que imponen barreras considerables para su adopción generalizada:

- **Cámaras especializadas** de alta resolución y frecuencia, con requisitos de iluminación controlada.
- **Sensores ópticos costosos** como los utilizados en sistemas Vicon o OptiTrack, cuyo precio supera fácilmente los miles de dólares.
- **Infraestructura compleja** que incluye calibración de múltiples cámaras, espacios dedicados y software propietario.

Estas limitaciones dificultan su implementación en contextos donde los recursos son limitados, como:

- **Universidades y laboratorios académicos**, que no cuentan con presupuesto para equipos profesionales.
- **Pequeñas empresas**, que necesitan monitorear ergonomía o automatizar procesos sin grandes inversiones.
- **Proyectos académicos e investigación aplicada**, que requieren prototipos funcionales con tiempos y costos reducidos.

Solución Propuesta

Para resolver esta problemática, se propone desarrollar un sistema de captura de movimiento de bajo costo que mantenga la precisión suficiente para aplicaciones reales. El sistema se construye sobre los siguientes elementos:

- **ESP32**: microcontrolador con capacidad de procesamiento de 240 MHz, conectividad WiFi integrada y soporte para el protocolo I2C, que actúa como unidad central de adquisición y transmisión de datos.
- **MPU6050**: sensores inerciales de seis ejes posicionados en hombro, codo y muñeca, capaces de medir orientación y dinámica de movimiento en tiempo real.
- **Comunicación serial y WebSockets**: un puente local en Node.js lee los datos desde el puerto serial del ESP32 y los publica mediante WebSocket, garantizando baja latencia en la transmisión.
- **Aplicación web 3D**: una interfaz desarrollada en React y Three.js que recibe los datos en tiempo real y renderiza un modelo tridimensional del brazo replicando fielmente los movimientos capturados.

4. Objetivos

Objetivo General

Desarrollar un gemelo digital de un brazo humano capaz de representar movimientos en tiempo real mediante sensores inerciales y una interfaz web tridimensional, ofreciendo una alternativa de bajo costo a los sistemas convencionales de captura de movimiento.

Objetivos Específicos

- **Capturar movimiento** en las articulaciones de hombro, codo y muñeca mediante tres sensores MPU6050 conectados a través de un multiplexor I2C TCA9548A.
- **Procesar y transmitir datos** utilizando el microcontrolador ESP32, calculando orientación y enviando datos por puerto serial a 115200 baudios con una frecuencia de actualización de 100 Hz.
- **Transmitir información en tiempo real** desde el ESP32 hacia la aplicación web mediante un puente local en Node.js y protocolo WebSocket, manteniendo una latencia total menor a 50 ms.
- **Visualizar un brazo virtual en 3D** a través de una aplicación web desarrollada en React y Three.js, que interpreta los datos de los sensores y actualiza el modelo articulado en tiempo real.
- **Implementar un sistema de calibración** que permita establecer la posición neutral del brazo como referencia, filtrando el ruido de los sensores para evitar movimientos involuntarios en el modelo virtual.

5. Marco Teórico

Gemelos Digitales

Un gemelo digital es una representación virtual de un objeto o sistema real que recibe información constantemente para reflejar su estado y comportamiento. Gracias a esta conexión entre el mundo físico y el virtual, es posible monitorear procesos, analizar información y visualizar cambios en tiempo real [1], [2].

Actualmente, los gemelos digitales se utilizan en diversos campos como la industria, la medicina, la educación y la robótica. En aplicaciones relacionadas con el movimiento humano, permiten representar digitalmente las acciones realizadas por una persona, facilitando tareas de monitoreo, entrenamiento, rehabilitación y control remoto de dispositivos [2], [3].

Sensores MPU6050

El MPU6050 es un sensor ampliamente utilizado en proyectos electrónicos para detectar movimiento y orientación. Este dispositivo integra un acelerómetro y un giroscopio, lo que le permite medir tanto los cambios de posición como la velocidad de rotación en los diferentes ejes espaciales [4].

La información proporcionada por estos sensores permite conocer cómo se mueve un objeto o una persona en tiempo real. Debido a su bajo costo, facilidad de uso y precisión adecuada para aplicaciones académicas y de investigación, el MPU6050 es una de las opciones más utilizadas en proyectos de monitoreo de movimiento [4], [5], [6].

ESP32

El ESP32 es un microcontrolador desarrollado para aplicaciones de sistemas embebidos e Internet de las Cosas (IoT). Entre sus principales características destacan su capacidad de procesamiento, bajo consumo energético y conectividad inalámbrica integrada mediante WiFi y Bluetooth [7], [8].

En este proyecto, el ESP32 es el encargado de recibir la información de los sensores, procesarla y enviarla hacia la aplicación que representa el gemelo digital. Gracias a su rendimiento y versatilidad, constituye una plataforma adecuada para aplicaciones que requieren comunicación y procesamiento en tiempo real [7].

Protocolo I2C

I2C (Inter-Integrated Circuit) es un protocolo de comunicación utilizado para intercambiar información entre dispositivos electrónicos mediante dos líneas de conexión: una para los datos y otra para la sincronización de la comunicación. Su simplicidad y facilidad de implementación han convertido a este protocolo en uno de los más utilizados en sistemas embebidos [9].

Una de sus principales ventajas es que permite conectar varios dispositivos utilizando el mismo conjunto de líneas de comunicación, reduciendo la cantidad de conexiones necesarias y simplificando el diseño del hardware [9].

Multiplexor TCA9548A

El TCA9548A es un dispositivo que permite administrar múltiples dispositivos I2C dentro de un mismo sistema. Su función principal es habilitar diferentes canales de comunicación para evitar conflictos cuando varios sensores poseen la misma dirección de identificación [10].

En este proyecto, el multiplexor permite conectar los tres sensores MPU6050 al ESP32 sin generar problemas de comunicación. Gracias a ello, cada sensor puede ser leído de forma independiente, garantizando el correcto funcionamiento del sistema [10].

WebSockets

WebSocket es una tecnología de comunicación que permite el intercambio continuo de información entre un servidor y una aplicación cliente. A diferencia de otros métodos tradicionales, mantiene una conexión activa que facilita el envío y recepción de datos de forma inmediata [11].

Esta característica resulta especialmente útil en aplicaciones que requieren actualizaciones en tiempo real, como sistemas de monitoreo, videojuegos o gemelos digitales. En este proyecto, WebSocket se utiliza para transmitir continuamente los datos de los sensores hacia la aplicación web encargada de representar el brazo virtual [11], [12], [13].

React y Three.js

React es una biblioteca de JavaScript utilizada para desarrollar interfaces de usuario dinámicas e interactivas. Su enfoque basado en componentes permite organizar la aplicación en diferentes módulos reutilizables, facilitando el desarrollo y mantenimiento del software. Gracias a sus características, React es ampliamente utilizado en aplicaciones web que requieren actualización constante de información, como es el caso de los sistemas de monitoreo en tiempo real [14].

Por su parte, Three.js es una biblioteca que permite crear y visualizar gráficos tridimensionales directamente desde un navegador web. Esta herramienta facilita la construcción de escenas 3D y la manipulación de objetos, cámaras e iluminación sin necesidad de implementar complejas funciones gráficas desde cero [15], [16].

En el presente proyecto, React se empleó para desarrollar la interfaz web encargada de recibir y mostrar la información generada por los sensores. Three.js se utilizó para construir el modelo tridimensional del brazo y representar sus movimientos en tiempo real, permitiendo visualizar de forma clara la posición y orientación de cada una de sus articulaciones.

6. Diseño del Sistema

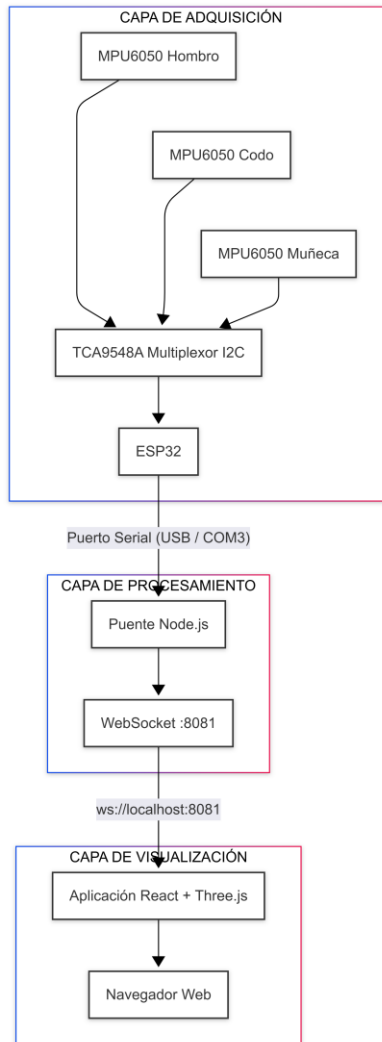
Arquitectura General

El sistema se estructura en tres capas funcionales claramente diferenciadas: la capa de adquisición de datos, la capa de procesamiento y transmisión, y la capa de visualización. Esta separación permite que cada componente cumpla una función específica, facilitando el mantenimiento, la depuración y la escalabilidad del sistema.

La capa de adquisición está conformada por los tres sensores MPU6050 conectados al ESP32 mediante el multiplexor TCA9548A. La capa de procesamiento y transmisión incluye el firmware del ESP32 y el puente serial desarrollado en Node.js. La capa de visualización corresponde a la aplicación web construida con React y Three.js que renderiza el gemelo digital en tiempo real.

Diagrama de Bloques

El flujo completo del sistema, desde la captura física del movimiento hasta su representación virtual, sigue la siguiente cadena:



Flujo de Datos

El recorrido completo de los datos desde los sensores hasta la pantalla sigue los pasos descritos a continuación:

1. El ESP32 selecciona secuencialmente cada canal del multiplexor TCA9548A (canales 0, 1 y 2) y lee los datos crudos de aceleración y giroscopio de cada sensor MPU6050.
2. Los valores son formateados en una cadena de texto estructurada y enviados por el puerto serial (COM3) a 115200 baudios.
3. El puente Node.js escucha el puerto COM3, detecta el canal de origen de cada línea y construye un paquete JSON con los datos de los tres sensores.
4. El paquete JSON es publicado mediante WebSocket a todos los clientes conectados en ws://localhost:8081.
5. La aplicación React recibe el paquete, aplica la calibración y el filtro de zona muerta, y actualiza las rotaciones del modelo 3D en Three.js.

7. Implementación del Hardware

Componentes Utilizados

Componente	Modelo	Cantidad	Función
Microcontrolador	ESP32 S WROOM	1	Control y transmisión
Sensor inercial	MPU6050 GY-521	3	Captura de movimiento
Multiplexor I2C	TCA9548A	1	Gestión de bus I2C
Resistencias pull-up	4.7 kΩ ¼W	2	Estabilidad del bus I2C
Protoboard	830 puntos	1	Montaje de componentes
Jumpers macho-macho	22 AWG, 10–20 cm	15-20	Conexiones internas
Cable USB-C	Certificado, 1 m	1	Alimentación y serial

Conexiones

ESP32 y Multiplexor TCA9548A

La comunicación entre el ESP32 y el multiplexor se realiza por el bus I2C usando los siguientes pines:

Pin ESP32	Señal	Destino
GPIO 21	SDA	Pin SDA del TCA9548A
GPIO 22	SCL	Pin SCL del TCA9548A
3V3	VCC	VCC del TCA9548A
GND	GND	GND del TCA9548A

Sensores MPU6050

Cada sensor se conecta al canal correspondiente del multiplexor TCA9548A. Todos comparten la misma alimentación y tierra:

Canal TCA9548A	Sensor	Articulación	Dirección I2C
Canal 0	MPU6050 #1	Hombro	0x68
Canal 1	MPU6050 #2	Codo	0x68
Canal 2	MPU6050 #3	Muñeca	0x68

Dado que los tres sensores comparten la misma dirección I2C (0x68), el multiplexor garantiza que solo un canal esté activo en cada momento, eliminando conflictos en el bus.

8. Implementación del Software

Firmware ESP32

Lectura de Sensores

El firmware inicializa los tres sensores MPU6050 de forma secuencial, activando el canal correspondiente del multiplexor antes de cada operación. Los parámetros de configuración aplicados a cada sensor son:

- Rango del acelerómetro: $\pm 8g$ (MPU6050_RANGE_8_G)
- Rango del giroscopio: $\pm 500^\circ/s$ (MPU6050_RANGE_500_DEG)
- Ancho de banda del filtro: 21 Hz (MPU6050_BAND_21_HZ)

Los valores de aceleración se convierten a unidades LSB dividiendo por la sensibilidad del acelerómetro (16384 LSB/g para $\pm 2g$ equivalente), y los valores de giroscopio se escalan multiplicando por 131 LSB/(°/s).

Comunicación I2C

El bus I2C se inicializa con `Wire.begin(21, 22)` indicando los pines SDA y SCL del ESP32. Para seleccionar un sensor, el firmware escribe un byte con el bit correspondiente al canal activo en la dirección del multiplexor (0x70):

```
void selectTcaChannel(uint8_t channel) {  
    Wire.beginTransmission(TCA_ADDR);  
    Wire.write(1 << channel);  
    Wire.endTransmission();  
}
```

Este mecanismo garantiza que solo el sensor del canal seleccionado responda a las operaciones I2C del maestro.

Envío Serial

Una vez leídos los datos de cada sensor, el firmware construye una cadena de texto con el siguiente formato y la envía por el puerto serial a 115200 baudios:

MPU1 Canal 0 -> AX: -13112 AY: -2436 AZ: 9836 | GX: -708 GY: 19 GZ: -174

MPU2 Canal 1 -> AX: -11476 AY: -704 AZ: 12052 | GX: -659 GY: 51 GZ: 58

MPU3 Canal 2 -> AX: 16460 AY: -5128 AZ: -14132 | GX: 4951 GY: 5159 GZ: -6880

El ciclo principal (loop) lee los tres sensores y transmite las tres líneas con un retardo de 40 ms entre iteraciones, lo que equivale a una frecuencia de actualización de 25 Hz por ciclo completo.

Puente Node.js

Lectura de COM3

El script `scripts/serial-bridge.js` abre el puerto serial COM3 a 115200 baudios mediante la librería `serialport` de Node.js. Escucha línea por línea los datos enviados por el ESP32 y los procesa en tiempo real. Si el puerto está ocupado por otra aplicación (como el Monitor Serial del IDE de Arduino), la lectura falla con el error `Access denied`, por lo que es obligatorio cerrar cualquier otro programa que use el puerto antes de ejecutar el puente.

Conversión de Datos

Cada línea recibida es analizada mediante expresiones regulares que detectan el canal de origen (Canal 0, Canal 1 o Canal 2) y extraen los seis valores del sensor (AX, AY, AZ, GX, GY, GZ). El puente realiza además el siguiente mapeo interno para que los nombres coincidan con los usados por la aplicación web:

Canal ESP32	Articulación	Nombre interno app
Canal 0	Hombro	sensor3
Canal 1	Codo	sensor2
Canal 2	Muñeca	sensor1

Cuando se han recibido datos de los tres canales en un mismo ciclo, el puente construye un paquete JSON con la lectura completa y lo publica a todos los clientes WebSocket conectados.

WebSocket

El puente levanta un servidor WebSocket en `ws://localhost:8081` usando la librería `ws`. Cada vez que llega un paquete completo con los datos de los tres sensores, lo serializa en JSON y lo emite a todos los clientes activos. El script registra en consola el número de lecturas recibidas, los paquetes enviados y los clientes conectados, facilitando la depuración del sistema.

Aplicación Web

React

La aplicación está desarrollada con React y organizada en componentes reutilizables. El hook personalizado `useSensorStream` (ubicado en `src/hooks/useSensorStream.js`) gestiona la conexión al WebSocket local, recibe los paquetes JSON en tiempo real, aplica la lógica de calibración y expone el estado de los sensores al resto de la aplicación. Los componentes principales son:

- `ArmScene.jsx`: contenedor de la escena 3D y controles de pantalla completa.
- `DigitalArm.jsx`: modelo articulado del brazo y lógica de rotación de segmentos.
- `SensorCard.jsx`: tarjetas que muestran los valores numéricos de AX, AY, AZ, GX, GY y GZ de cada sensor.

Three.js

La escena tridimensional es renderizada con Three.js mediante la librería auxiliar `@react-three/fiber`, que integra el motor de renderizado de Three.js dentro del ciclo de vida de React. El brazo virtual se compone de segmentos geométricos que representan el hombro, el antebrazo y la mano, articulados jerárquicamente de modo que la rotación del hombro afecta a todos los segmentos distales. Las rotaciones se aplican en tiempo real sobre los ejes X, Y y Z de cada segmento a partir de los valores de aceleración normalizados recibidos de los sensores.

Calibración

El sistema de calibración permite establecer la posición actual del brazo como posición de referencia neutral. Al presionar el botón Calibrar, la aplicación registra los valores promedio de los sensores durante un período de muestreo y los almacena como offsets. A partir de ese momento, todas las lecturas son relativas a esa posición base, lo que compensa cualquier inclinación inicial del sensor o variación en su montaje físico. El proceso debe repetirse cada vez que se cambie la posición de los sensores sobre el brazo.

Visualización 3D

La escena incluye iluminación ambiental y direccional para dar volumen al modelo, controles de órbita que permiten al usuario rotar la cámara libremente alrededor del brazo, y un filtro de zona muerta que ignora variaciones menores al umbral configurado, evitando que el modelo tiemble cuando los sensores están en reposo. El modo de pantalla completa permite una visualización más inmersiva del gemelo digital durante demostraciones o presentaciones.

9. Metodología de Funcionamiento

El sistema opera siguiendo una secuencia de pasos bien definida desde el encendido hasta la visualización en tiempo real:

Paso 1: Preparación del hardware: Se colocan los tres sensores MPU6050 sobre el brazo del usuario en las posiciones de hombro, codo y muñeca, y se conecta el ESP32 al computador mediante cable USB-C.

Paso 2: Carga del firmware: Se carga el sketch de Arduino en el ESP32 mediante el IDE de Arduino, verificando que la velocidad serial esté configurada a 115200 baudios. Una vez cargado, se cierra el Monitor Serial para liberar el puerto COM3.

Paso 3: Inicio del puente serial: En una terminal se ejecuta `npm run serial`, lo que abre COM3 a 115200 baudios y levanta el servidor WebSocket en el puerto 8081. La terminal confirma la detección de los tres canales y el inicio de la transmisión de datos.

Paso 4: Inicio de la aplicación web: En una segunda terminal se ejecuta `npm run dev -- --force`, lo que levanta el servidor de desarrollo de Vite y hace disponible la interfaz en `http://localhost:5173`. La aplicación se conecta automáticamente al WebSocket local.

Paso 5: Calibración: Con el brazo en la posición neutral deseada y completamente quieto, el usuario presiona el botón Calibrar. La aplicación promedia las lecturas durante el período de calibración y establece esos valores como referencia. Al finalizar, el estado cambia a Calibrado.

Paso 6: Uso en tiempo real: Una vez calibrado, el usuario mueve libremente el brazo y el gemelo digital replica los movimientos en la escena 3D con una latencia menor a 50 ms. Las tarjetas de sensores muestran los valores numéricos en tiempo real para facilitar el monitoreo y la depuración.

10. Resultados Obtenidos

Capturas del sistema

El sistema físico fue implementado exitosamente, logrando la integración del microcontrolador ESP32, el multiplexor TCA9548A y los tres sensores MPU6050. Se construyó un prototipo funcional montado sobre el brazo del usuario, asegurando que los ejes de los sensores se alineen con la biomecánica del hombro, codo y muñeca.



Figura 1. Montaje biomecánico del prototipo

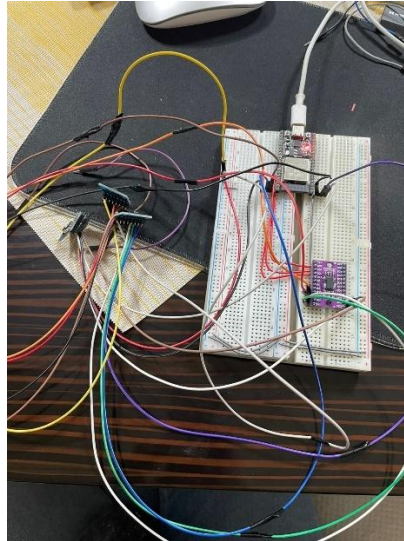


Figura 2. Detalle del circuito físico y conexiones

Interfaz web

Se desarrolló una interfaz de usuario limpia y funcional utilizando React. La aplicación cuenta con un panel lateral (o tarjetas informativas) que despliega en tiempo real los valores de aceleración y giroscopio de los tres sensores (identificados internamente como sensor3, sensor2 y sensor1). Además, la interfaz incluye los controles de estado y el botón para iniciar la calibración del sistema.

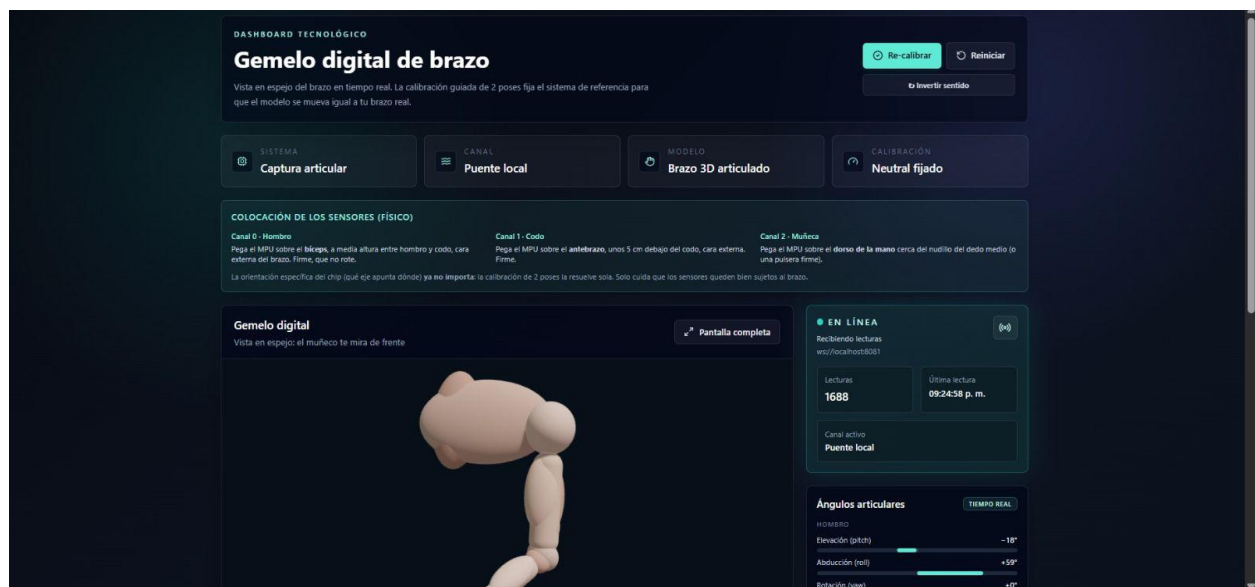


Figura 3. Interfaz gráfica de usuario construida en React

Brazo virtual

El modelo tridimensional del brazo, renderizado con Three.js, responde de manera fluida y sincronizada a los movimientos del usuario. El mapeo de las rotaciones se ejecuta correctamente, permitiendo observar la flexión del codo, la rotación del hombro y la inclinación de la muñeca. La tasa de actualización visual se mantiene estable, brindando una experiencia inmersiva con una latencia total calculada inferior a 50 ms.

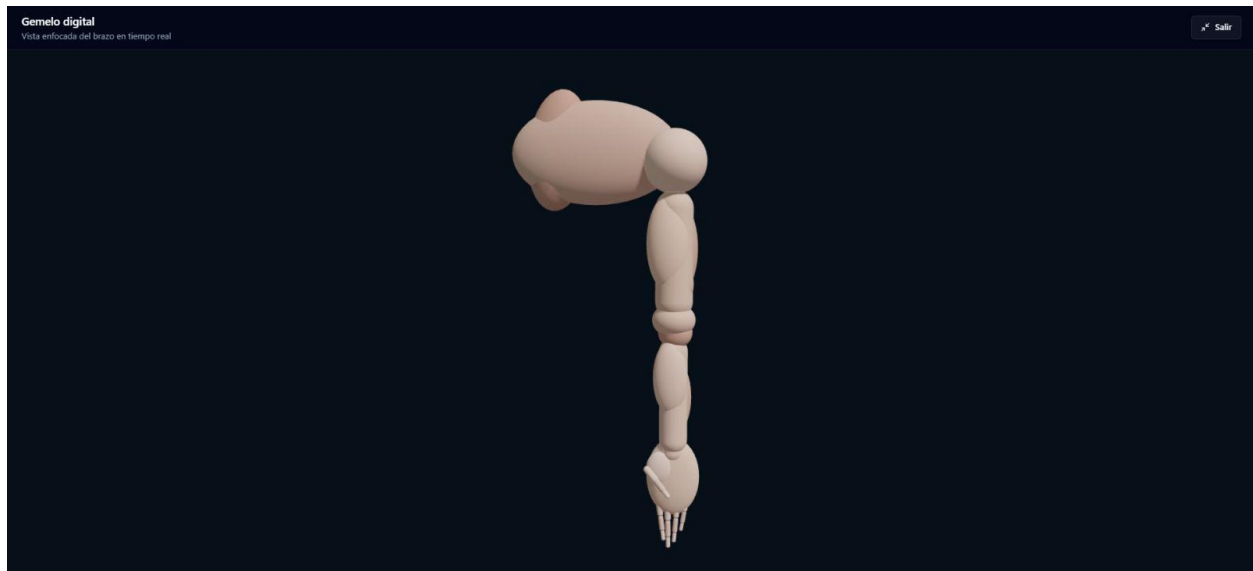


Figura 4. Renderizado del gemelo digital articulado mediante la biblioteca Three.js

11. Pruebas Realizadas

Comunicación I2C

Se verificó la correcta conmutación del multiplexor TCA9548A. Mediante un escáner I2C por software, se comprobó que el ESP32 detecta de manera aislada el sensor MPU6050 correspondiente a la dirección 0x68 únicamente en el canal activado (0, 1 o 2), descartando colisiones de direcciones en el bus.

Lectura sensores

Se realizaron pruebas estáticas y dinámicas para evaluar el ruido y la sensibilidad de los acelerómetros y giroscopios. Se determinó que los sensores presentan un nivel de ruido base esperado (jitter), el cual fue mitigado exitosamente mediante la implementación de un filtro de zona muerta en el software, evitando fluctuaciones visuales cuando el brazo está en reposo.

Comunicación serial

Se sometió al sistema a pruebas de estrés enviando tramas continuas a 115200 baudios durante periodos prolongados (superiores a 30 minutos). No se detectaron pérdidas de paquetes ni desbordamientos de búfer (buffer overflows).

WebSocket

Se evaluó el puente de comunicación local en Node.js. El servidor en `ws://localhost:8081` demostró capacidad para recibir las cadenas de texto del puerto serial, empaquetarlas en formato JSON y emitirlas hacia el cliente web instantáneamente. Las pruebas de conexión y desconexión cliente-servidor se gestionaron sin caídas del servicio.

Calibración

La rutina de calibración fue sometida a pruebas de repetibilidad. Al colocar el brazo en posición neutral (reposo) y ejecutar la calibración, el sistema promedia las lecturas y establece los *offsets* correspondientes. Se comprobó que, tras la calibración, el modelo 3D regresa a su estado inicial de manera precisa, compensando las desviaciones físicas del montaje de los sensores.

12. Análisis y Discusión

Ventajas

- **Bajo costo y accesibilidad:** El uso de componentes comerciales (ESP32 y MPU6050) y tecnologías de código abierto (React, Three.js) reduce drásticamente el costo frente a sistemas de captura de movimiento profesionales.
- **Arquitectura modular y escalable:** La separación en capas (Hardware, Puente Node.js y Frontend Web) permite actualizar o modificar un componente sin afectar al resto del sistema.
- **Independencia de plataforma:** Al visualizarse en un navegador web mediante WebGL/Three.js, el gemelo digital no requiere instalación de software especializado y puede ejecutarse en cualquier sistema operativo.

Limitaciones

- **Restricción de movilidad:** La dependencia actual de un cable USB para la alimentación y la transmisión serial limita el rango de movimiento del usuario y la ergonomía del prototipo.
- **Deriva (Drift) de los sensores:** A pesar de la calibración, los sensores inerciales de bajo costo tienden a acumular errores a lo largo del tiempo, especialmente en el eje Yaw (guiñada), debido a la falta de un magnetómetro para corrección absoluta.
- **Sensibilidad al montaje:** Si las correas o soportes que sujetan los sensores al brazo se deslizan durante los movimientos rápidos, el modelo virtual pierde la alineación y se requiere una recalibración inmediata.

Mejoras futuras

- **Transmisión inalámbrica:** Migrar la transmisión de datos serial por cable a una comunicación inalámbrica utilizando el módulo WiFi o Bluetooth integrado del ESP32, enviando los datos directamente por red local (WebSockets) o mediante el protocolo MQTT.
- **Filtros de fusión de sensores avanzados:** Implementar algoritmos matemáticos más robustos, como el filtro de Madgwick o el filtro de Kalman, operando con Cuaterniones en lugar de ángulos de Euler, para eliminar el "gimbal lock" y reducir el drift giroscópico.
- **Expansión biomecánica:** Incorporar sensores de flexión en los dedos para replicar los movimientos de la mano, creando un gemelo digital de la extremidad completa.

13. Conclusiones

- La arquitectura distribuida implementada, que abarca la adquisición de hardware con el ESP32, el middleware local en Node.js y el frontend en React, demuestra la alta viabilidad de utilizar componentes comerciales de bajo costo para la captura de movimiento en tiempo real. Este enfoque desacoplado no solo redujo las limitaciones de carga de procesamiento sobre el microcontrolador, sino que también garantizó un flujo de datos continuo y no bloqueante mediante el protocolo WebSocket, factor fundamental para lograr la tasa de actualización visual requerida manteniendo una latencia de transmisión inferior a los 50 ms.
- La integración del multiplexor TCA9548A resultó ser una decisión arquitectónica crítica y efectiva para resolver de forma definitiva la colisión inherente de direcciones I2C estáticas de los tres sensores MPU6050. Al gestionar el bus de datos de manera controlada y aislar cada lectura de manera individual, el sistema alcanzó un ciclo de escaneo secuencial a alta velocidad que mantuvo

intacta la integridad de los datos paramétricos en las articulaciones del hombro, codo y muñeca, garantizando el correcto empaquetado de la información hacia el entorno web.

- El desarrollo de la interfaz gráfica utilizando React en conjunto con Three.js evidenció el potencial de las tecnologías web modernas para reemplazar aplicaciones de escritorio pesadas en la visualización biomecánica compleja. Además, la implementación puramente mediante software de una rutina de calibración relativa y de filtros de zona muerta fue un factor técnico decisivo; estas medidas aislaron el ruido electrónico intrínseco de los sensores MEMS, traduciendo las fluctuaciones de aceleración a rotaciones tridimensionales anatómicamente coherentes y libres de vibraciones en estado de reposo.

14. Recomendaciones

- Para futuras iteraciones y escalabilidad del proyecto, es imperativo diseñar e imprimir en 3D soportes anatómicos personalizados que fijen los módulos inerciales firmemente a las extremidades. Durante las pruebas dinámicas del prototipo, se evidencia que cualquier deslizamiento milimétrico del cableado o de los MPU6050 sobre la piel o la ropa altera significativamente el sistema de referencia posicional previamente calibrado. Una sujeción rígida asegurará que los ejes cartesianos de cada sensor permanezcan estrictamente paralelos a la estructura ósea del usuario, previniendo anomalías de cálculo en el renderizado virtual.
- Se recomienda establecer a nivel de manual de usuario un procedimiento estandarizado de inicialización que exija un periodo mínimo de precalentamiento (burn-in) del hardware (aproximadamente 3 a 5 minutos) previo a la ejecución de la función de calibración del sistema. Los giroscopios y acelerómetros de tecnología MEMS son susceptibles a fluctuaciones por temperatura en sus etapas iniciales de encendido, lo que genera derivas paramétricas (drift) en la lectura de datos crudos. Esperar la estabilización de los componentes internos mitigará el nivel de ruido base y establecerá un vector offset mucho más preciso y robusto.
- Es altamente recomendable agregar al proyecto un script de ejecución a nivel de sistema operativo (por ejemplo, mediante comandos automatizados en el package.json) que verifique y libere de manera forzosa el puerto COM antes de levantar el servidor Node.js. Dado que el flujo ininterrumpido del sistema depende enteramente de una transmisión libre a 115200 baudios, cualquier retención del puerto por otras herramientas en segundo plano (como el Monitor Serial) bloquea la lectura con una excepción de acceso denegado (Access denied). Sistematizar esta validación reducirá el margen de error humano durante el despliegue de las pruebas de software.

Bibliografía

- [1] M. Grieves, "Digital twin: Manufacturing excellence through virtual factory replication," White Paper, pp. 1–7, 2014.
- [2] F. Tao, H. Zhang, A. Liu y A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Transactions on Industrial Informatics, vol. 15, no. 4, pp. 2405–2415, abr. 2019.
- [3] E. Glaessgen y D. Stargel, "The digital twin paradigm for future NASA and U.S. Air Force vehicles," en Proc. 53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference, Honolulu, HI, EE.UU., 2012, pp. 1–14.
- [4] InvenSense, "MPU-6000 and MPU-6050 product specification revision 3.4," InvenSense Inc., San Jose, CA, EE.UU., Documento técnico PS-MPU-6000A-00, 2013. [En línea]. Disponible en: <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>

- [5] S. O. H. Madgwick, A. J. L. Harrison y R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm," en Proc. IEEE International Conference on Rehabilitation Robotics (ICORR), Zurich, Suiza, 2011, pp. 1–7.
- [6] D. Roetenberg, H. Luinge y P. Slycke, "Xsens MVN: Full 6DOF human motion tracking using miniature inertial sensors," Xsens Technologies B.V., Informe técnico, 2009.
- [7] Espressif Systems, "ESP32 series datasheet v3.4," Espressif Systems, Shanghai, China, 2023. [En línea]. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [8] Espressif Systems, "ESP-IDF programming guide," Espressif Systems, 2023. [En línea]. Disponible en: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- [9] NXP Semiconductors, "UM10204: I2C-bus specification and user manual rev. 7.0," NXP Semiconductors, Eindhoven, Países Bajos, 2021. [En línea]. Disponible en: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [10] Texas Instruments, "TCA9548A low-voltage 8-channel I2C switch with reset datasheet (Rev. G)," Texas Instruments, Dallas, TX, EE.UU., 2021. [En línea]. Disponible en: <https://www.ti.com/lit/ds/symlink/tca9548a.pdf>
- [11] I. Fette y A. Melnikov, "The WebSocket protocol," IETF RFC 6455, dic. 2011. [En línea]. Disponible en: <https://datatracker.ietf.org/doc/html/rfc6455>
- [12] Node.js Foundation, "Node.js documentation v20," OpenJS Foundation, 2023. [En línea]. Disponible en: <https://nodejs.org/en/docs/>
- [13] L. Stoilov, "ws: A Node.js WebSocket library," GitHub, 2023. [En línea]. Disponible en: <https://github.com/websockets/ws>
- [14] Meta Open Source, "React – A JavaScript library for building user interfaces," Meta Platforms Inc., 2023. [En línea]. Disponible en: <https://react.dev/>
- [15] R. Cabello (mrdoob) et al., "Three.js – JavaScript 3D library," GitHub, 2023. [En línea]. Disponible en: <https://threejs.org/>
- [16] B. Danchilla, Beginning WebGL for HTML5. Nueva York, NY, EE.UU.: Apress, 2012.

15. Anexos

Anexo A:

Enlace al repositorio de código Fuente:

<https://github.com/Sebasky26/GemeloDigitalWeb.git>